



國立臺灣科技大學
**TAIWAN
TECH** National Taiwan University of
Science and Technology

Neural Network-Based Fault Identification and Classification Using RBFNN and FNN Models

Advisor: Prof. Ryan Lian

Presenter: Prayoga & Abdillah

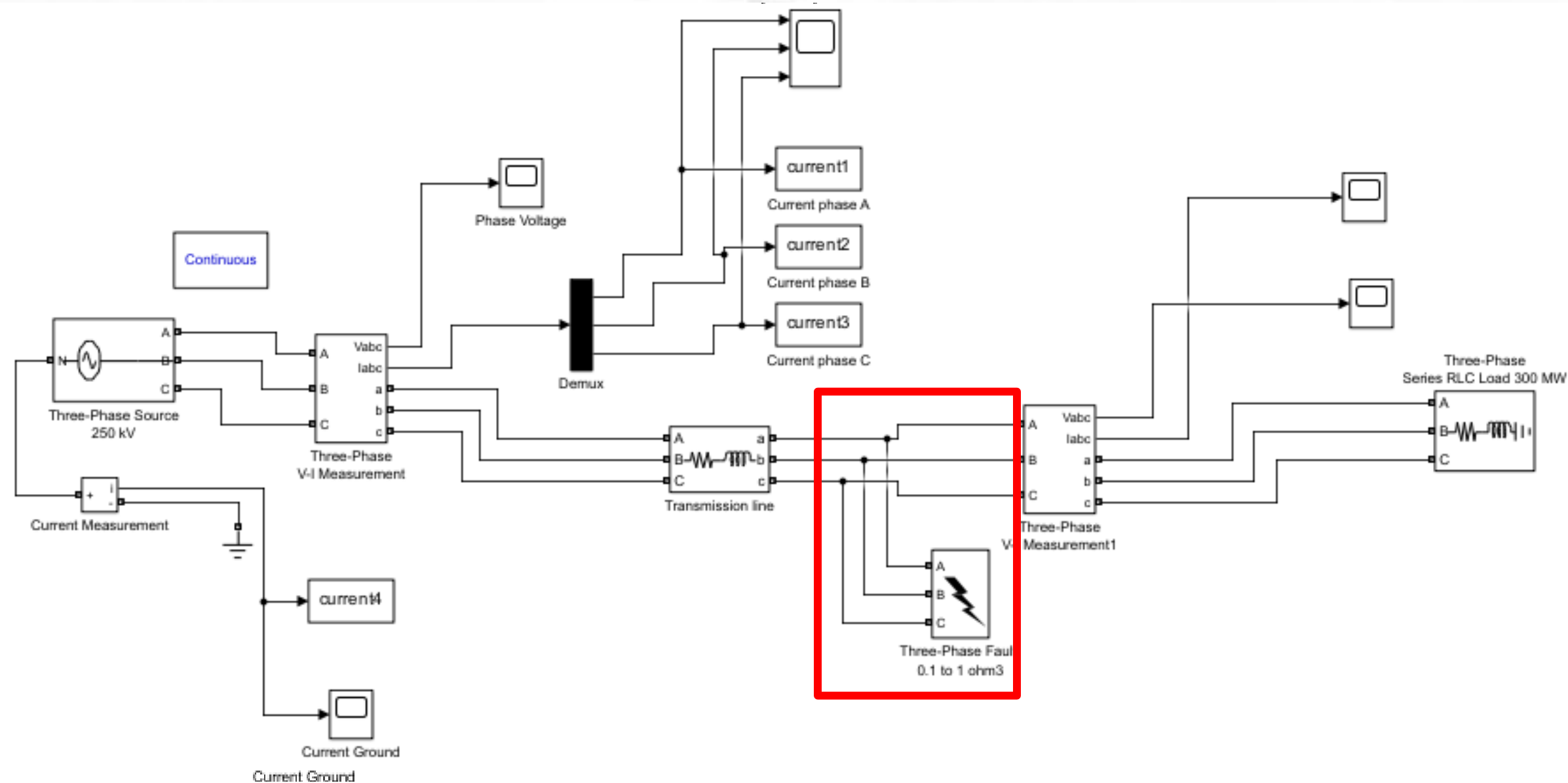
Date: 2024/08/28

Learned Materials:

1. Time series forecasting
2. Exploratory Data Analysis (EDA)
3. Hyperparameter
4. Feature Engineering



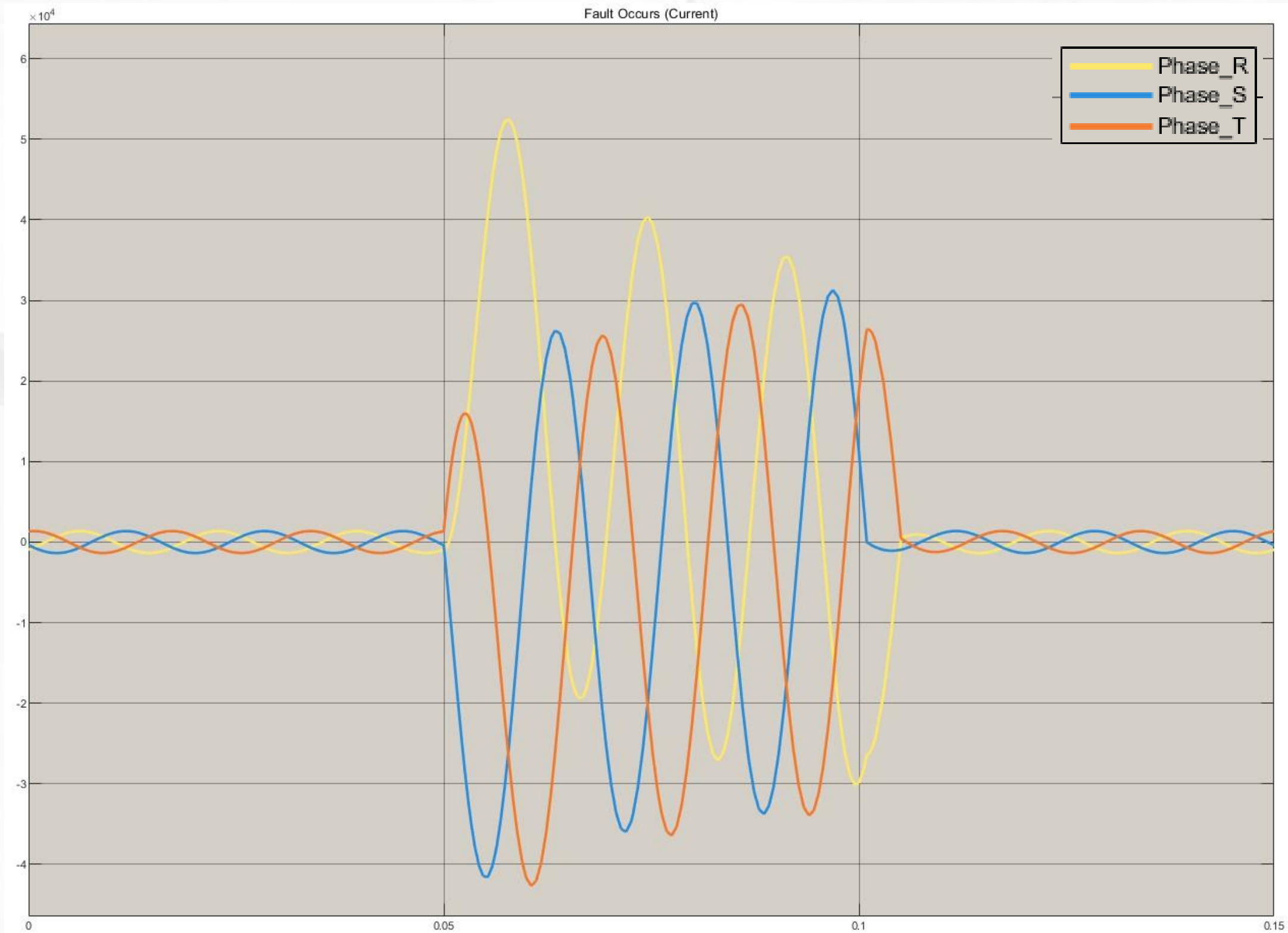
Fault Identification in Three-Phase Transmission Line



- **Objective:**
to predict max coefficient of each phase during fault conditions.
- **Fault ground resistance:**
0.1 – 1 ohm (0.1 ohm increment)
- **Type of fault :**
 - RST-G - R-T
 - RST - S-T
 - RS-G - R-G
 - RT-G - S-G
 - ST-G - T-G
 - R-B - No fault
- **Fault Location:**
 - Before transmission line
 - After transmission line
- **Output value for each phase:**
 - 1 = fault
 - 0 = no fault

Note: In high-voltage systems, even a small fault resistance can result in extremely large fault currents. Since the wavelet transform captures the energy and intensity of these currents, the coefficients can be quite large as well.

Current waveforms during fault condition:



Wavelet transform

```
% Extract current signals from workspace
currentA = evalin('base', 'current1');
currentB = evalin('base', 'current2');
currentC = evalin('base', 'current3');
currentG = evalin('base', 'current4');
```

```
% Perform wavelet decomposition
[cA, LA] = wavedec(currentA, 1, 'db4');
[cB, LB] = wavedec(currentB, 1, 'db4');
[cC, LC] = wavedec(currentC, 1, 'db4');
[cG, LG] = wavedec(currentG, 1, 'db4');
```

```
% Extract detailed coefficients
coefA = detcoef(cA, LA, 1);
coefB = detcoef(cB, LB, 1);
coefC = detcoef(cC, LC, 1);
coefG = detcoef(cG, LG, 1);
```



Data X (Input)

Data y (Output)

No	Fault ground Resistance Ω	Fault Location	Type of Fault	Max. Coefficient of Phase R Current	Max. Coefficient of Phase S Current	Max. Coefficient of Phase T Current	Max. Coefficient of Ground Current	Output Value for Phase R	Output Value for Phase S	Output Value for Phase T	Output Value for Ground
1	0.1	Before transmission line	ABC-G	16097055.71	40725475.36	16097055.71	79893.30	1	1	1	1
2	0.1	Before transmission line	ABC	16097055.71	40725475.36	16097055.71	0.01	1	1	1	0
3	0.1	Before transmission line	AB-G	10803035.42	20464068.14	103.98	86421.48	1	1	0	1
4	0.1	Before transmission line	AC-G	20261370.35	103.98	8624427.56	202614.07	1	0	1	1
5	0.1	Before transmission line	BC-G	103.98	40725475.36	7176136.72	99363.73	0	1	1	1
6	0.1	Before transmission line	A-B	10794282.03	20362761.11	103.98	0.01	1	1	0	0
7	0.1	Before transmission line	A-C	20362677.38	103.98	8615308.28	0.02	1	0	1	0
8	0.1	Before transmission line	B-C	103.98	40725475.36	7257318.23	0.01	0	1	1	0
9	0.1	Before transmission line	A-G	159569.04	103.98	103.98	170252.99	1	0	0	1
10	0.1	Before transmission line	B-G	103.98	403268.91	128.92	129747.00	0	1	0	1
11	0.1	Before transmission line	C-G	103.98	103.98	160779.45	403222.06	0	0	1	1
12	0.1	Before transmission line	NO FAULT	103.98	103.98	103.98	0.00	0	0	0	0
.....											
228	0.9	After transmission line	NO FAULT	72.11	132.09	115.72	0.00	0	0	0	0
229	1	After transmission line	ABC-G	767.31	700.11	792.80	415.11	1	1	1	1
230	1	After transmission line	ABC	919.83	976.99	728.80	0.01	1	1	1	0
231	1	After transmission line	AB-G	1086.16	675.75	85.05	585.48	1	1	0	1
232	1	After transmission line	AC-G	488.89	131.47	840.88	515.19	1	0	1	1
233	1	After transmission line	BC-G	133.09	508.17	689.76	249.11	0	1	1	1
234	1	After transmission line	A-B	502.80	527.85	114.10	0.01	1	1	0	0
235	1	After transmission line	A-C	373.92	99.41	488.20	0.01	1	0	1	0
236	1	After transmission line	B-C	115.72	594.60	735.53	0.01	0	1	1	0
237	1	After transmission line	A-G	852.78	75.29	102.68	778.38	1	0	0	1
238	1	After transmission line	B-G	114.87	421.49	107.07	659.07	0	1	0	1
239	1	After transmission line	C-G	131.07	131.07	846.38	591.62	0	0	1	1
240	1	After transmission line	NO FAULT	72.11	132.09	115.72	0.00	0	0	0	0

Total data : 240 data

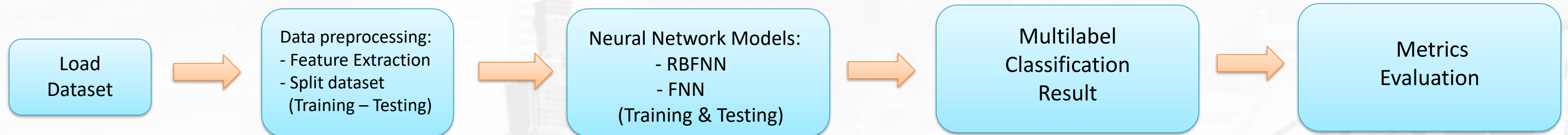
Output value for each phase:

1 = fault

0 = no fault



MATLAB Workflow for Multilabel Classification



X (input)				y (output)			
16097055.71	40725475.36	16097055.71	79893.30	1	1	1	1
16097055.71	40725475.36	16097055.71	0.01	1	1	1	0
10803035.42	20464068.14	103.98	86421.48	1	1	0	1
20261370.35	103.98	8624427.56	202614.07	1	0	1	1
103.98	40725475.36	7176136.72	99363.73	0	1	1	1
10794282.03	20362761.11	103.98	0.01	1	1	0	0
20362677.38	103.98	8615308.28	0.02	1	0	1	0
103.98	40725475.36	7257318.23	0.01	0	1	1	0
159569.04	103.98	103.98	170252.99	1	0	0	1
103.98	403268.91	128.92	129747.00	0	1	0	1
103.98	103.98	160779.45	403222.06	0	0	1	1
103.98	103.98	103.98	0.00	0	0	0	0
767.31	700.11	792.80	415.11	1	1	1	1
919.83	976.99	728.80	0.01	1	1	1	0
1086.16	675.75	85.05	585.48	1	1	0	1
488.89	131.47	840.88	515.19	1	0	1	1
133.09	508.17	689.76	249.11	0	1	1	1
502.80	527.85	114.10	0.01	1	1	0	0
373.92	99.41	488.20	0.01	1	0	1	0
115.72	594.60	735.53	0.01	0	1	1	0
852.78	75.29	102.68	778.38	1	0	0	1
114.87	421.49	107.07	659.07	0	1	0	1
131.07	131.07	846.38	591.62	0	0	1	1
72.11	132.09	115.72	0.00	0	0	0	0

Split Data:
Training 70% = 168
Testing 30 % = 72

```

I = trainX';
T = trainY';
goal = 0.0;
spread = 1;
net = newrb(I, T, goal, spread);

P = testX';
Y = sim(net, P);
predictedY = double(Y' >= 0.5); % Convert to binary

% Display predicted vs actual data for RBFNN
disp('RBFNN Model: Predicted vs Actual (Testing Data)');
disp('Predicted Y:');
disp(predictedY);
disp('Actual Y:');
disp(testY);

% Evaluate the RBFNN model using classification metrics
confMat = confusionmat(testY(:), predictedY(:));
accuracy = sum(diag(confMat)) / sum(confMat(:));
precision = diag(confMat) ./ sum(confMat, 2);
recall = diag(confMat) ./ sum(confMat, 1);
f1Score = 2 * (precision .* recall) ./ (precision + recall);
  
```

```

% Step 4: Train and test the FNN model
I = trainX';
T = trainY';
hiddenLayerSize = 10;
net = feedforwardnet(hiddenLayerSize);

net.trainParam.goal = 0.0;
net.trainParam.epochs = 1000;
net.trainParam.showWindow = true;

[net, tr] = train(net, I, T);

P = testX';
Y = net(P);
predictedY = double(Y' >= 0.5); % Convert to binary

% Display predicted vs actual data for FNN
disp('FNN Model: Predicted vs Actual (Testing Data)');
disp('Predicted Y:');
disp(predictedY);
disp('Actual Y:');
disp(testY);

% Evaluate the FNN model using classification metrics
confMat = confusionmat(testY(:), predictedY(:));
accuracy = sum(diag(confMat)) / sum(confMat(:));
precision = diag(confMat) ./ sum(confMat, 2);
recall = diag(confMat) ./ sum(confMat, 1);
f1Score = 2 * (precision .* recall) ./ (precision + recall);
  
```

Multilabel Classification Result

RBFINN model

RBFNN Model: Predicted vs				Actual Y:			
Predicted Y:							
0	1	1	0	0	1	1	0
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0
1	1	1	1	0	0	1	1
0	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1
0	0	0	0	0	0	0	0
1	1	1	0	1	1	1	0
0	1	1	0	0	1	1	0
1	1	1	1	1	1	0	1
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	0	1
1	1	1	1	0	0	1	1
1	1	1	1	0	1	0	1
1	1	1	1	0	1	0	1
1	1	1	1	0	1	1	0
0	1	1	0	1	1	0	0
1	1	0	0	1	1	1	0
1	1	1	0	1	0	1	0
1	0	1	0	0	0	1	1
1	1	1	1	0	1	0	1
1	1	1	1	1	0	0	1
1	1	1	1	0	1	1	1
1	1	1	1	1	1	1	1

```
RBFFNN Model Evaluation Metrics:
Accuracy: 0.82292
Precision: 0.796
Recall: 0.88084
F1 Score: 0.80422
>>
```

FNN model

```
>> FNNmodel
FNN Model: Predicted vs Actual Y:
Predicted Y:
```

0	1	1	1
1	1	1	1
1	1	1	1
0	0	1	1
0	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
0	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	0	0
1	1	1	0
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
0	1	1	1
1	1	1	0
1	1	1	1
1	0	1	1
1	1	1	1

```
Actual Y:
```

0	1	1	0
1	1	1	1
0	0	0	0
0	0	1	1
0	1	1	0
1	0	0	1
0	0	0	0
1	1	1	0
0	1	1	0
1	1	0	1
0	0	1	1
0	1	0	1
0	1	0	1
0	1	1	0
1	1	0	0
1	1	1	0
1	0	1	0
0	0	1	1
0	1	0	1
1	0	0	1
0	1	1	1
1	1	1	1
0	0	0	0
0	1	0	1
1	1	1	1

```
FNN Model Evaluation Metrics:
Accuracy: 0.64236
Precision: 0.59546
Recall: 0.70474
F1 Score: 0.55935
>>
```



THANK YOU

